Laboratório 3 - Criação de Threads (PThreads)

Aluno: Maria Eduarda da Nóbrega

Os três programas foram compilados com gcc -Wall <arquivo>.c -o <arquivo> -lpthread e executados em Linux. Os códigos comentados estão nos arquivos thread-create.c, thread-join.c e thread-print.c. As saídas citadas abaixo são de execuções reais; como a ordem depende do escalonador, ela pode mudar a cada execução.

Exercício 1 - thread-create.c

1. Código comentado

```
/*
 * Exercício 1 - Criação de threads POSIX (thread-create.c)
 * Compilar: gcc -Wall thread-create.c -o thread-create -lpthread
 * Base: exemplo de Carlos Maziero (DINF/UFPR), comentado por João Victor Guimarães.
 */
#include <pthread.h>   // pthread_create, pthread_exit, pthread_t
#include <stdio.h>     // printf, perror
#include <stdlib.h>    // exit
#include <unistd.h>    // sleep

#define NUM_THREADS 16 // quantidade de threads que a main vai criar

// Corpo executado por cada thread. Recebe um ponteiro genérico (void *)
// que aqui carrega o número da thread, e devolve void * (nada, no caso).
void *threadBody (void *id)
{
  long tid = (long) id ;                // converte o argumento de volta para inteiro

  printf ("t%02ld: Olá!\n", tid) ;      // anuncia que a thread começou
  sleep (3) ;                           // bloqueia por 3 s: a CPU fica livre para as outras
  printf ("t%02ld: Tchau!\n", tid) ;    // anuncia que vai terminar
  pthread_exit (NULL) ;                 // encerra somente esta thread, sem valor de retorno
}

int main (int argc, char *argv[])
{
  pthread_t thread [NUM_THREADS] ;      // vetor com os identificadores das threads
  long i, status ;                      // i: contador / id; status: retorno de pthread_create

  for (i=0; i<NUM_THREADS; i++)
  {
    printf ("Main: criando thread %02ld\n", i) ;
    // cria a thread: guarda o ID em thread[i], atributos padrão (NULL),
    // função a executar (threadBody) e argumento (o índice i, convertido p/ void *)
    status = pthread_create (&thread[i], NULL, threadBody, (void *) i) ;
    if (status)                         // retorno diferente de 0 indica erro
    {
      perror ("pthread_create") ;
      exit (1) ;
    }
  }
  printf ("Main: fim\n") ;
  // encerra só a thread main; o processo continua vivo até as demais terminarem.
  // (um return/exit aqui mataria o processo inteiro e as threads)
  pthread_exit (NULL) ;
```

```
}
```

Trecho da saída

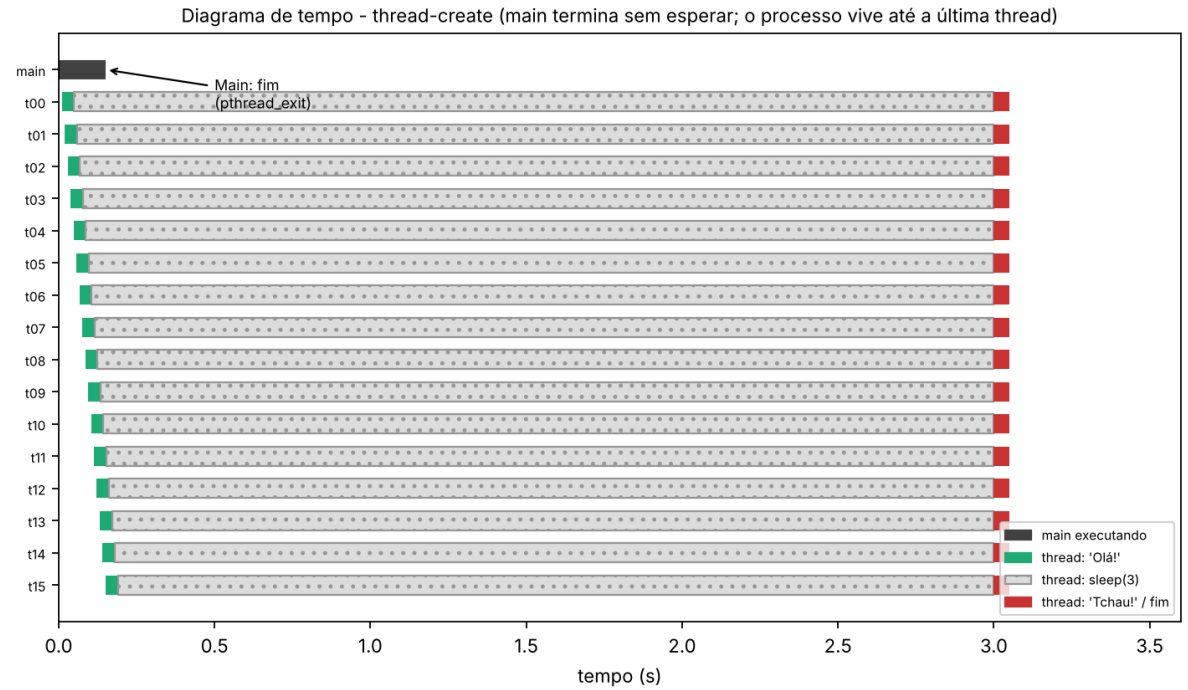
```
Main: criando thread 00
Main: criando thread 01
Main: criando thread 02
t00: Olá!
t01: Olá!
t02: Olá!
Main: criando thread 03
Main: criando thread 04
t03: Olá!
Main: criando thread 05
...
Main: criando thread 15
t14: Olá!
Main: fim
t15: Olá!
t01: Tchau!
t02: Tchau!
t09: Tchau!
...
```

2. A ordem de criação, ativação e encerramento é a mesma?

Não. A criação é sequencial e determinística, pois quem cria é a thread main dentro de um laço: 0, 1, 2, ..., 15. Já a ativação (o momento em que cada thread recebe a CPU pela primeira vez) é decidida pelo escalonador do sistema operacional. Depois de pthread_create a thread fica pronta, mas não há garantia de que execute logo. Na saída se vê isso: t00 imprimiu "Olá!" antes de a main criar a thread 03, e "Main: fim" apareceu antes do "Olá!" da t15. O encerramento também não segue a ordem de criação: todas dormem 3 s e acordam quase ao mesmo tempo, e quem imprime "Tchau!" primeiro é quem o escalonador colocar na CPU antes (na execução feita a ordem foi t01, t02, t09, t00, t11, ...). Resumindo: a ordem de criação é fixa, enquanto a ordem de ativação e de encerramento é não determinística, pois as threads concorrem pela CPU.

Obs.: a main chama pthread_exit no final, o que encerra apenas a thread main. Se usasse return ou exit, o processo inteiro terminaria e as threads seriam mortas antes do "Tchau!".

3. Diagrama de tempo



Exercício 2 - thread-join.c

1. Código comentado

```
/*
 * Exercício 2 - Threads POSIX com pthread_join (thread-join.c)
 * Compilar: gcc -Wall thread-join.c -o thread-join -lpthread
 * Base: exemplo de Carlos Maziero (DINF/UFPR), comentado por João Victor Guimarães.
 */
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define NUM_THREADS 16

void *threadBody (void *id)
{
  long tid = (long) id ;                // número da thread recebido da main

  printf ("t%02ld: Olá!\n", tid) ;
  sleep (3) ;                           // simula trabalho / espera
  printf ("t%02ld: Tchau!\n", tid) ;
  pthread_exit (NULL) ;                 // termina; o estado fica disponível para o join
}

int main (int argc, char *argv[])
{
  pthread_t thread [NUM_THREADS] ;
  pthread_attr_t attr ;                 // objeto de atributos das threads
  long i, status ;

  // inicializa attr com valores padrão e declara as threads como JOINABLE,
  // ou seja, a main poderá esperar por elas e recuperar seu término
```

```
  pthread_attr_init (&attr) ;
  pthread_attr_setdetachstate (&attr, PTHREAD_CREATE_JOINABLE) ;

  // fase 1: cria todas as threads (passando attr)
  for (i=0; i<NUM_THREADS; i++)
  {
    printf ("Main: criando thread %02ld\n", i) ;
    status = pthread_create (&thread[i], &attr, threadBody, (void *) i) ;
    if (status)
    {
      perror ("pthread_create") ;
      exit (1) ;
    }
  }

  // fase 2: espera cada thread terminar, em ordem (0, 1, 2, ...)
  // pthread_join bloqueia a main até a thread indicada encerrar
  for (i=0; i<NUM_THREADS; i++)
  {
    printf ("Main: aguardando thread %02ld\n", i);
    status = pthread_join (thread[i], NULL) ;   // NULL: não queremos o valor de retorno
    if (status)
    {
      perror ("pthread_join") ;
      exit (1) ;
    }
  }
  printf ("Main: fim\n") ;               // só chega aqui depois de TODAS terminarem
  pthread_attr_destroy (&attr) ;         // libera o objeto de atributos
  pthread_exit (NULL) ;
}
```

Trecho da saída

```
Main: criando thread 15
t14: Olá!
Main: aguardando thread 00
t15: Olá!
t01: Olá!
t00: Tchau!
t02: Tchau!
t06: Tchau!
t08: Tchau!
t10: Tchau!
t12: Tchau!
t14: Tchau!
t01: Tchau!
t03: Tchau!
t04: Tchau!
t05: Tchau!
t07: Tchau!
t09: Tchau!
Main: aguardando thread 01
...
Main: aguardando thread 13
Main: aguardando thread 14
Main: aguardando thread 15
Main: fim
```

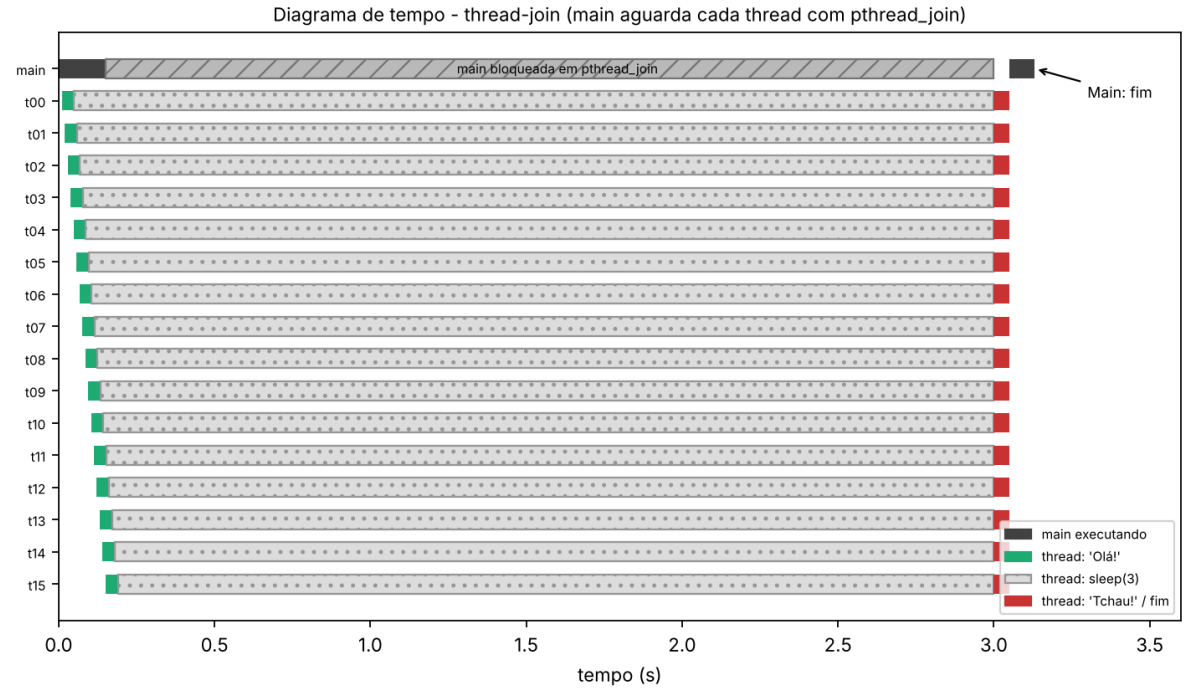
2. Objetivo de attr e de pthread_join

attr (pthread_attr_t) é um objeto de atributos passado ao pthread_create para definir como a thread será criada. Aqui ele é iniciado com pthread_attr_init e configurado com

pthread_attr_setdetachstate(PTHREAD_CREATE_JOINABLE), isto é, as threads são criadas no estado "joinable": outra thread pode esperar seu término e coletar seu estado. (No Linux esse já é o padrão, mas o código deixa explícito.) No fim, pthread_attr_destroy libera o objeto.

pthread_join(thread[i], NULL) bloqueia a thread que o chamou (a main) até que a thread indicada termine. Isso tem três efeitos: (a) sincroniza, pois a main só imprime "Main: fim" depois que as 16 threads acabaram; (b) libera os recursos da thread terminada (sem o join, uma thread joinable finalizada continua ocupando recursos, como um zumbi); (c) permite recuperar o valor de retorno (aqui NULL, pois não interessa). Diferente do Exercício 1, a main não encerra antes das filhas: ela espera por elas, em ordem (0, 1, ..., 15). Como todas dormem o mesmo tempo, os joins seguintes retornam quase imediatamente, e na saída as mensagens "Main: aguardando thread 06...15" aparecem seguidas.

3. Diagrama de tempo



Exercício 3 - thread-print.c

1. Código comentado

```
/*
 * Exercício 3 - Threads POSIX com variável global compartilhada (thread-print.c)
 * Compilar: gcc -Wall thread-print.c -o thread-print -lpthread
 * Base: exemplo de Carlos Maziero (DINF/UFPR), comentado por João Victor Guimarães.
 */
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define NUM_THREADS 16

int x = 0 ;   // variável GLOBAL: fica no espaço de endereçamento do processo,
```

```
              // portanto é a MESMA para todas as threads (compartilhada)

void *threadBody (void *id)
{
  long tid = (long) id ;

  x++ ;                                          // incrementa o x compartilhado
  printf ("t%02ld: Olá!   (x=%02d)\n", tid, x) ;
  sleep (3) ;                                    // enquanto dorme, as outras threads rodam
  x++ ;                                          // incrementa de novo
  printf ("t%02ld: Tchau! (x=%02d)\n", tid, x) ;
  pthread_exit (NULL) ;
}

int main (int argc, char *argv[])
{
  pthread_t thread [NUM_THREADS] ;
  long i, status ;

  for (i=0; i<NUM_THREADS; i++)
  {
    printf ("Main: criando thread %02ld\n", i) ;
    status = pthread_create (&thread[i], NULL, threadBody, (void *) i) ;
    if (status)
    {
      perror ("pthread_create") ;
      exit (1) ;
    }
  }
  printf ("Main: fim\n") ;
  pthread_exit (NULL) ;
}
```

Trecho da saída

```
Main: criando thread 08
t07: Olá!   (x=07)
t01: Olá!   (x=08)
t08: Olá!   (x=09)
Main: criando thread 09
Main: criando thread 10
t09: Olá!   (x=10)
Main: criando thread 11
...
t14: Olá!   (x=15)
Main: fim
t15: Olá!   (x=16)
t00: Tchau! (x=17)
t02: Tchau! (x=18)
t01: Tchau! (x=19)
t08: Tchau! (x=20)
t03: Tchau! (x=21)
...
```

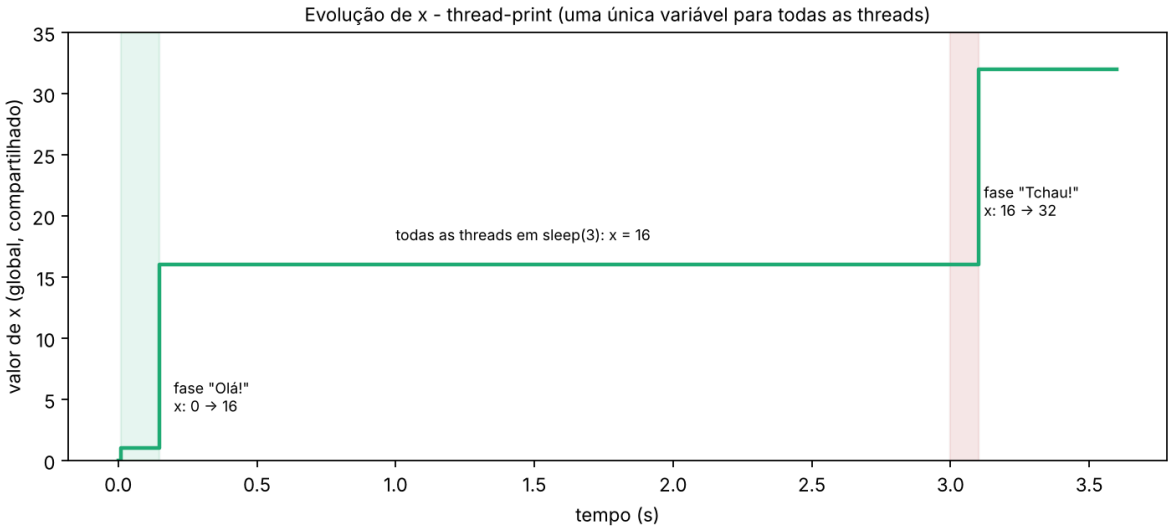
2. Evolução de x com threads versus com fork

Com threads, x é uma variável global e todas as threads de um processo compartilham o mesmo espaço de endereçamento, então existe um único x. Cada thread incrementa duas vezes (uma antes do "Olá!" e outra antes do "Tchau!"), logo x cresce continuamente de 0 até 16 x 2 = 32, como se vê na saída (x=01 ... x=16 nos "Olá!" e x=17 ... x=32 nos "Tchau!"). O valor impresso por uma thread depende do que as outras já fizeram.

Em um programa equivalente com fork, cada filho recebe uma cópia do espaço de endereçamento do pai (com copy-on-write). Cada processo teria seu próprio x, começando em 0: todo filho imprimiria x=1 no "Olá!" e x=2 no "Tchau!", e o x do pai continuaria 0, pois os incrementos dos filhos não são visíveis para ninguém fora deles. Para trocar dados, os processos precisariam de IPC (pipes, memória compartilhada etc.).

Ressalva: x++ não é atômico (lê, soma, grava). Sem sincronização (mutex), duas threads podem ler o mesmo valor e uma atualização se perder (condição de corrida), e o valor final poderia ser menor que 32. Na execução feita deu 32, mas isso não é garantido; o esperado é proteger x com pthread_mutex_t.

3. Diagrama de tempo com a evolução de x



Resumo: nos instantes iniciais (≈ 0 a 0,15 s) as 16 threads são criadas e x vai de 0 a 16; durante o sleep(3) x fica parado em 16; ao acordarem (≈ 3 s) as threads incrementam de novo e x vai de 16 a 32.